

CS595 (Tuesday and 9 AM of the Class)

COMPUTER SCIENCE CAPSTONE COURSE

San Francisco Bay University, Fremont, CA

MIDTERM

Fall 2025



Prof. Ahmed Banafa

TWOS (TravelWithOutStress)

October 14th, 2025

Team Members

Umair Dada (Team Leader)	159855
Sangmin Lee	162342
Ayoyimika Ajibade	144210

Table of Contents

ABSTRACT	3
1. FRONTEND WORKS (UMAIR).....	4
2. NLP MODULE (SANGMIN)	9
3. APIS FOR SEARCH SERVICE (SANGMIN)	11
4. DATABASE AND BACKEND API MANAGEMENT (AYO).....	14
5. CLOUD DEPLOYMENT AND INFRASTRUCTURE (AYO).....	21
CONCLUSION	23
REFERENCES	24

Abstract.

An AI-powered travel planner would allow people to simplify the process of organizing trips. Expedia provides part of the solution, but the main problem remains, which humans have to search for each component of trips. By combining natural language understanding with real-time data from external APIs, users can enter requirements in natural language, and the NLP module in the system parses inputs, validates, and revises details, and coordinates with the backend to search flights, hotels, car rentals, and attractions. The backend manages the NLP module and external APIs communication, and data storage, while the frontend provides a user-friendly interface for presenting results.

1. Frontend Works (Umailr)

Our project **TWOS (Travel Without Stress)** needed a frontend that felt real, not like a prototype.

My part was to design and build the full user interface and everything the user touches and sees.

The goal was simple: make travel planning through chat feel natural and easy.



When the app opens, users see the TWOS logo with the tagline “Travel Without Stress.” Then an onboarding screen appears. It explains what the app does and ends with a button that says “Start Chatting.” The screens use clean fonts, smooth gradients, and simple layouts. The focus is on clarity, not decoration.

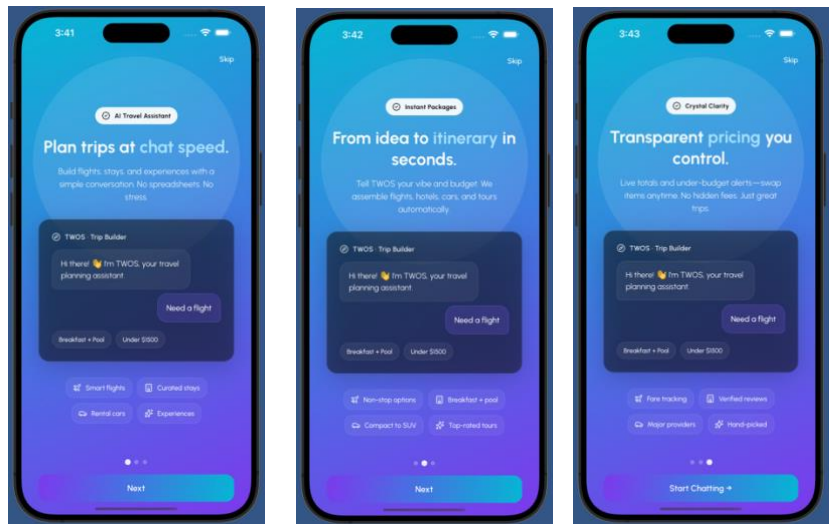


Figure 1 Splash screen and onboarding slide



Figure 2 Chat interface with typing animation

The chat screen is the main part of my work. Users type messages, and TWOS replies instantly. The chat scrolls automatically, and when the system is thinking, a small airplane animation shows inside a bubble. This makes the experience feel alive without needing real AI yet.

The chat can read what the user says and figure out what information is missing. For example, if someone writes “I want to fly to Doha in November,” the app knows it doesn’t have the departure city. It replies, “Where will you be flying from?” Then it shows small buttons like “San Francisco (SFO)” or

“Los Angeles (LAX).” Users can tap one or type their own answer.

We created small local files that store these questions and answers. One file holds the bot’s possible follow-up messages, another keeps quick options, and a third saves short greetings. This way, the frontend can handle most of the conversation by itself. Even without the backend, the app feels complete.

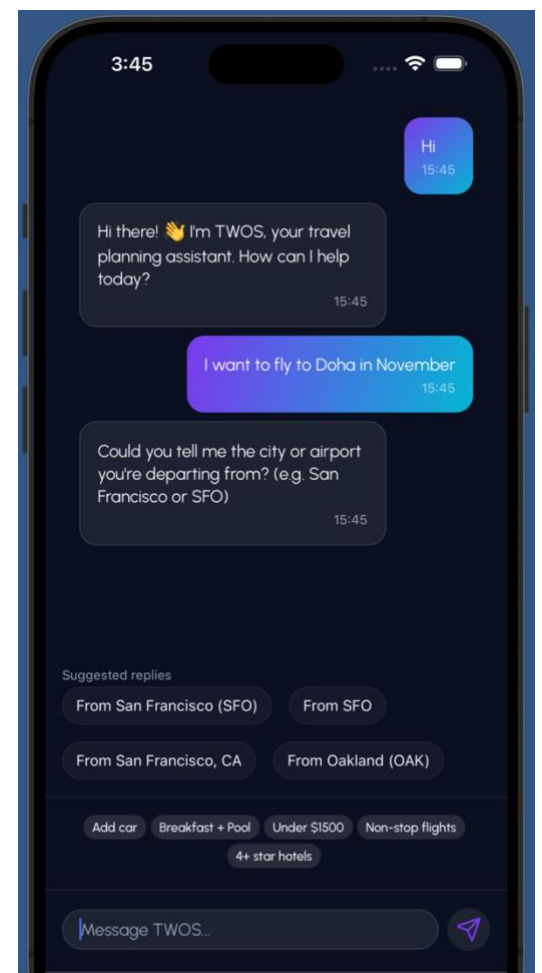


Figure 3 Showing the chat bubble saying “Where will you be flying from?” with suggestion chips under it.

The app also responds to simple greetings. If someone says “Hi” or “Hello,” TWOS answers right away with friendly messages like “Hi there! I’m TWOS, your travel assistant.” It doesn’t send these to the server. This keeps the chat fast and natural. The same logic works for travel intents. If a user says “I want to fly,” the app immediately asks “Where would you like to fly from?” instead of waiting for a backend reply.

To connect everything, we built a small test API that sends back sample data. When a user sends a message, the app sends JSON data to this server. The server replies with two things: what details are already known (called “slots”) and what details are missing. The app reads that reply, updates the conversation, and continues asking questions until all details are filled. Once the backend is ready, it can replace this test API without changing the frontend code.

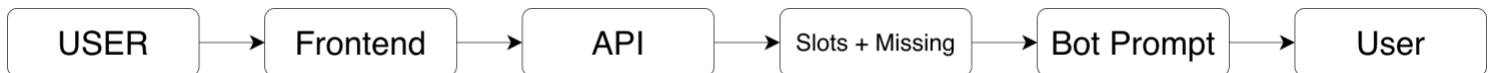


Figure 4 Data flow diagram

We also built preview screens for travel providers. The **Booking.com** page looks almost like the real site. It uses the same blue color and layout. The **Expedia** page uses gold and dark blue. These previews help users see what the final booking pages will look like. There’s also a fake Apple Pay checkout screen. It shows how payment will appear later when real integration is added.

Figure 5 Booking.com checkout preview

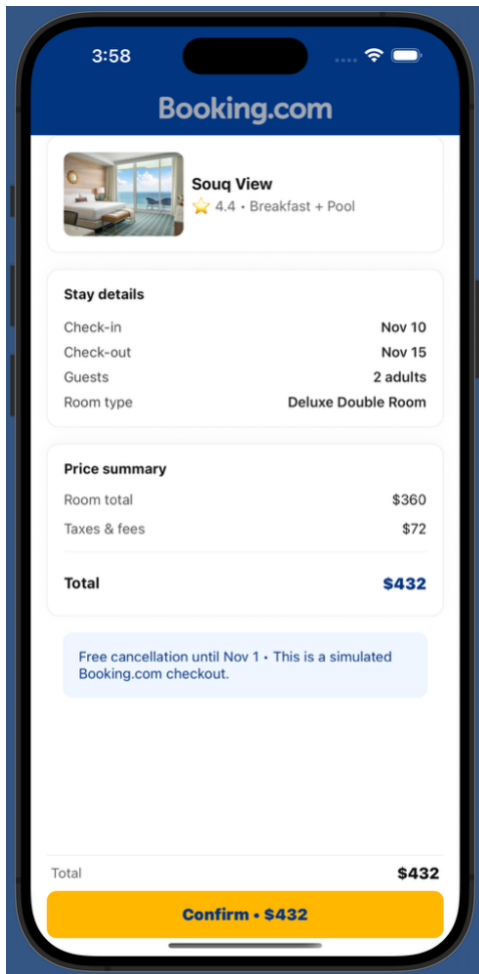
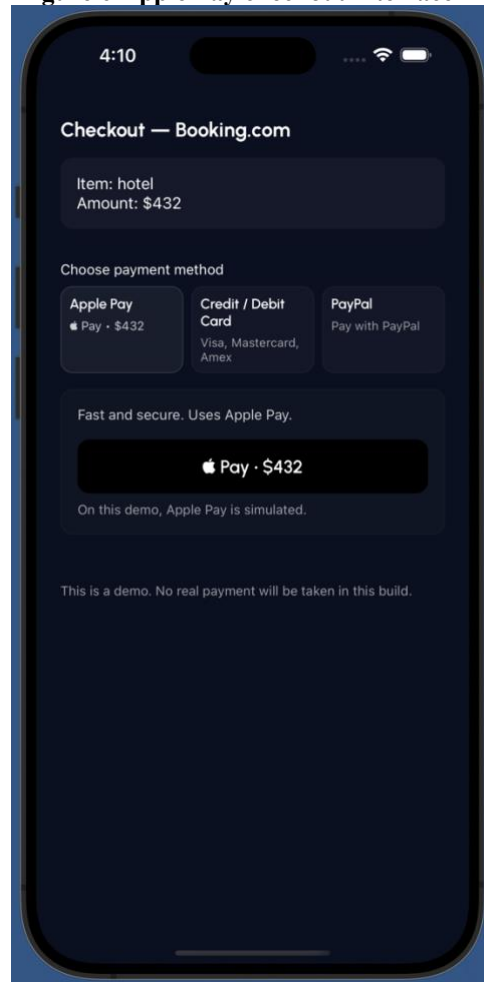


Figure 6 Apple Pay checkout interface



The hardest part of my work was getting the chat logic right. It had to respond smoothly, ask smart follow-up questions, and never break the flow. Building it meant thinking about how people actually talk. We kept sentences short, avoided robotic wording, and made sure each bot question sounded natural.

In the end, the frontend turned out solid. It covers the full trip planning flow from greeting the user to showing a final travel package. It looks consistent across iOS and Android, runs smoothly in Expo Go, and matches the brand idea of “Travel Without Stress.” Even without a live backend, the demo feels real and easy to use.

Working on the frontend taught me that design and clarity matter as much as logic. A simple, working interface can explain the idea better than technical notes ever could. My part made sure our team's concept came to life visually. When users try TWOS, they understand it instantly that's what makes this work worth it.

2. NLP Module (Sangmin)

For development efficiency, we separate the backend and the NLP module parts. Currently, the NLP module provides two services – chat and search. The chat service interacts with Frontend, and it has three major functionalities. First, if a slot ID is empty, it implies that the user starts a new chat, and then it assigns a unique slot ID. This slot ID is used to track slots during and after the chat. Second, parsing the user’s natural language and filling out the slots. Third, edit the slot information if the Frontend sends different information for the slot field. The following are samples of the chat service.

The screenshot displays a REST client interface with a POST request to `http://148.100.79.191:8000/chat`. The request body is a JSON object with the following structure:

```
1 {
2   "message": "Hi",
3   "current_slots": {
4     "slot_id": null,
5     "origin_airport_code": null,
6     "destination_airport_name": null,
7     "destination_airport_code": null,
8     "destination_city_code": null,
9     "dates": {
10      "start": null,
11      "end": null
12    },
13    "pax": {
14      "adults": 0,
15      "kids": 0
16    },
17    "budget": null,
18    "hotel": {
19      "request": null,
20      "amenities": [],
21      "rating": null
22    },
23    "car": null,
24    "attractions": []
25  }
26 }
```

The response body is a JSON object with the following structure:

```
1 {
2   "current_slots": {
3     "slot_id": "01K7DAA0BTWZJJTHDYY9HSR83",
4     "origin_airport_code": null,
5     "destination_airport_code": null,
6     "destination_city_name": null,
7     "destination_city_code": null,
8     "dates": {
9       "start": null,
10      "end": null
11    },
12    "pax": {
13      "adults": 1,
14      "kids": 0
15    },
16    "budget": null,
17    "hotel": {
18      "request": null,
19      "amenities": [],
20      "rating": null
21    },
22    "car": null,
23    "attractions": []
24  },
25   "missing": [
26     "origin_airport_code",
27     "destination_airport_code",
28     "destination_city_name",
29     "destination_city_code",
30     "dates",
31     "budget",
32     "hotel.request",
33     "car"
34   ]
35 }
```

NLP Module Figure 1. Initial request and response when the user starts a new chat

POST http://148.100.79.191:8000/chat 200 OK

Body Cookies Headers (4) Test Results

Params Authorization Headers (8) Body Scripts Settings

{} JSON Preview Visualize

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "message": "I am planning a family trip from SF to Seoul. There are 4 people, 2 adults and 2 kids.
3   From Nov 10 to Nov 25. I need a rental car during the trip. I don't need a hotel. My budget is $5k.",
4   "current_slots": {
5     "slot_id": "01K7DAA0BTWZJTHDYY9HSR83",
6     "origin_airport_code": null,
7     "destination_airport_code": null,
8     "destination_city_name": null,
9     "destination_city_code": null,
10    "dates": {
11      "start": null,
12      "end": null
13    },
14    "pax": {
15      "adults": 1,
16      "kids": 0
17    },
18    "budget": null,
19    "hotel": {
20      "request": null,
21      "amenities": [],
22      "rating": null
23    },
24    "car": null,
25    "attractions": []
26  },
27  "missing": [
28    "origin_airport_code",
29    "destination_airport_code",
30    "destination_city_name",
31    "destination_city_code",
32    "dates",
33    "budget",
34    "hotel.request",
35    "car"
36  ]
37 }

```

```

1 {
2   "current_slots": {
3     "slot_id": "01K7DAA0BTWZJTHDYY9HSR83",
4     "origin_airport_code": "SFO",
5     "destination_airport_code": "ICN",
6     "destination_city_name": "Seoul",
7     "destination_city_code": "SEL",
8     "dates": {
9       "start": "2025-11-10",
10      "end": "2025-11-25"
11    },
12    "pax": {
13      "adults": 2,
14      "kids": 2
15    },
16    "budget": 5000.0,
17    "hotel": {
18      "request": false,
19      "amenities": [],
20      "rating": null
21    },
22    "car": true,
23    "attractions": []
24  },
25  "missing": []
26 }

```

NLP Module Figure 2. Parsing the user's natural language and filling out the slots

POST http://148.100.79.191:8000/chat

Body Cookies Headers (4) Test Results

Params Authorization Headers (8) Body Scripts Settings

{} JSON Preview Visualize

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "message": "I want to leave from the San Jose instead of SF",
3   "current_slots": {
4     "slot_id": "01K78884QKMMEDZ4Z45K46CD",
5     "origin_airport_code": "SFO",
6     "destination_airport_code": "ICN",
7     "destination_city_name": "Seoul",
8     "destination_city_code": "SEL",
9     "dates": {
10      "start": "2025-11-10",
11      "end": "2025-11-25"
12    },
13    "pax": {
14      "adults": 2,
15      "kids": 2
16    },
17    "budget": 5000.0,
18    "hotel": {
19      "request": false,
20      "amenities": [],
21      "rating": null
22    },
23    "car": true,
24    "attractions": []
25  },
26  "missing": []
27 }

```

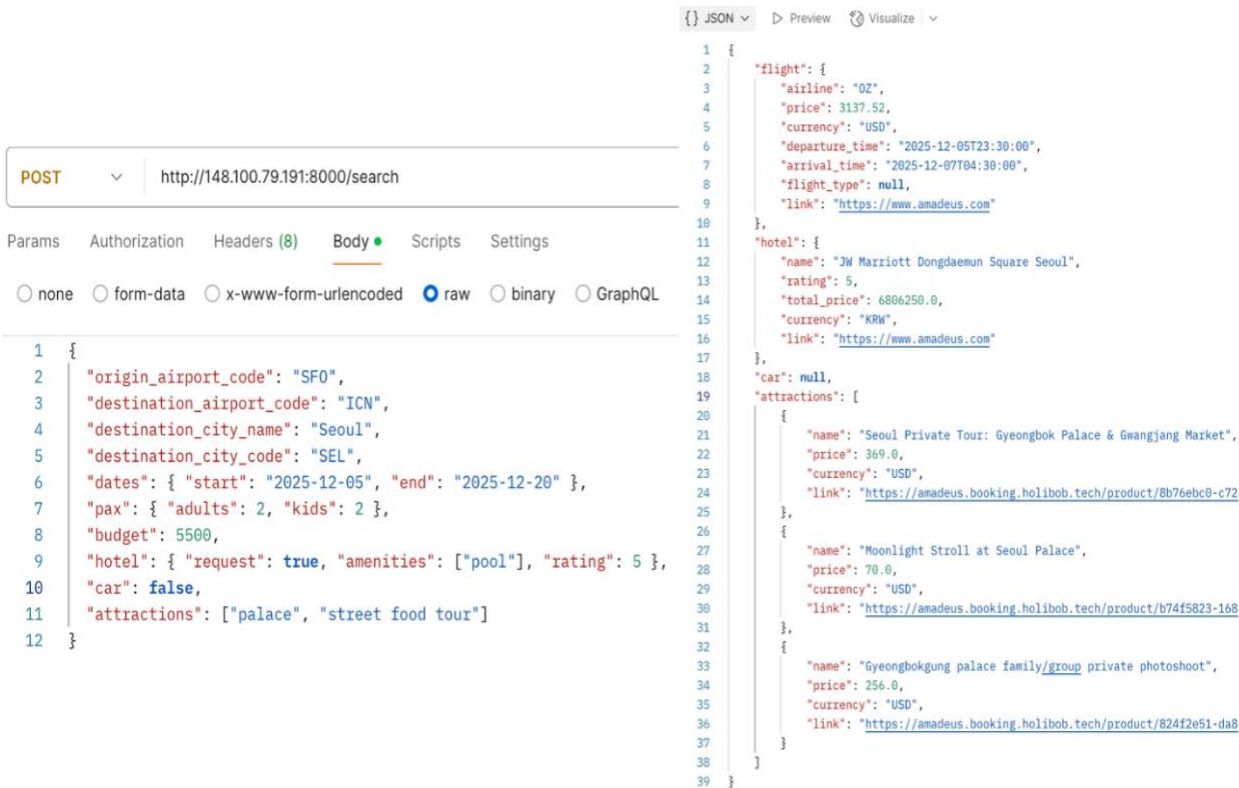
```

1 {
2   "current_slots": {
3     "slot_id": "01K78884QKMMEDZ4Z45K46CD",
4     "origin_airport_code": "SJC",
5     "destination_airport_code": "ICN",
6     "destination_city_name": "Seoul",
7     "destination_city_code": "SEL",
8     "dates": {
9       "start": "2025-11-10",
10      "end": "2025-11-25"
11    },
12    "pax": {
13      "adults": 2,
14      "kids": 2
15    },
16    "budget": 5000.0,
17    "hotel": {
18      "request": false,
19      "amenities": [],
20      "rating": null
21    },
22    "car": true,
23    "attractions": []
24  },
25  "missing": []
26 }

```

NLP Module Figure 3. Revise the slot field based on the user's request

The search service interacts with the backend and makes API calls for flights, hotels, rental cars, and attractions. It has two main functionalities. First, it makes API calls based on the slot information. Second, it makes an initial travel plan based on each API call's responses. The following is an example of the search service.



NLP Module Figure 4. Search service call APIs, organize them, and send the travel plan to FE.

3. APIs for Search service (Sangmin)

For search service APIs, we used the Amadeus API for flights, hotels, and attractions. It provides free and high-quality data in a test environment. We set basic principles for making travel plan using information from APIs.

- 1. Receive data from the APIs.

POST	/flight_search	Flight Search
POST	/hotel_search	Hotel Search
POST	/attractions_search	Attractions Search
POST	/car_search	Car Search

API Figure 1. Search API option list

2. Filter and/or sort data based on the users' request.
3. Hotels & Cars:
 - a. If the "request" field is false, then skip the related API calls.
4. Attractions:
 - a. Although the attractions field is empty, call the APIs for a recommendation.

By setting and following these principles, we are expecting not only better resource management, but also better trip plans based on the user's requests. The following are examples of how these principles are worked and applied.

<pre> ✓ Received slots for search: { "slot_id": "01K76JYPSGX40B08841VPETXRX", "origin_airport_code": "SJC", "destination_airport_code": "ICN", "destination_city_name": "Seoul", "destination_city_code": "SEL", "dates": { "start": "2025-11-10", "end": "2025-11-25" }, "pax": { "adults": 2, "kids": 2 }, "budget": 5000.0, "hotel": { "request": true, "amenities": ["breakfast"], "rating": 4 }, "car": false, "attractions": ["palace", "street food tour"] } </pre>	<pre> ✓ CAN SEARCH FLIGHTS: True ✓ ✈ Searching for flights... ✓ Loaded Amadeus Key: Yes ✓ Loaded Amadeus Secret: Yes { "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T18:20:00", "link": "https://www.amadeus.com" }, { "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T16:00:00", "link": "https://www.amadeus.com" }, { "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T18:20:00", "link": "https://www.amadeus.com" }, { "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T16:00:00", "link": "https://www.amadeus.com" }, { "airline": "KE", "price": 6141.12, "currency": "USD", "departure_time": "2025-11-10T06:20:00", "link": "https://www.amadeus.com" } ✈ Found cheapest flight: KE for \$5889.12 </pre>	Flight API sample 1
<pre> ✓ CAN SEARCH HOTELS: True ✓ 🏠 Searching for hotels... ✓ Loaded Amadeus Key: Yes ✓ Loaded Amadeus Secret: Yes 🏠 Found cheapest hotel: JW Marriott Dongdaemun Square Seoul and total price is \$7132950.0 </pre>	Hotel API sample 1	
<pre> ✓ CAN SEARCH ATTRACTIONS: True ✓ 📍 Searching for attractions... ✓ Loaded Amadeus Key: Yes ✓ Loaded Amadeus Secret: Yes ● Filtering attractions based on interests: ['palace', 'street food tour'] ✓ Found and processed 3 attractions. { "name": "Seoul Private Tour: Gyeongbok Palace & Gwangjang Market", "description": "", "rating": 4.5, "link": "https://amadeus.booking.holibob.tech/product/8b76ebc0-c727-4159-92e1-479ca738384e" }, { "name": "Moonlight Stroll at Seoul Palace", "description": "", "rating": 0.0, "price": 70.0, "link": "https://amadeus.booking.holibob.tech/product/b74f5823-168a-41fd-b465-4567423d0e6b" }, { "name": "Gyeongbokgung palace family/group private photoshoot", "description": "", "rating": 4.5, "link": "https://amadeus.booking.holibob.tech/product/824f2e51-da8a-4001-b44f-c2f9ecce5ee1" } 📍 Found 3 attractions. </pre>	Attractions API sample 1	

API Figure 2. Log to check/debug that APIs are working properly and following the principles 1

```

✓ Received slots for search: {
  "slot_id": "01K76JYP5GX4QB08841VPETXR",
  "origin_airport_code": "SJC",
  "destination_airport_code": "ICN",
  "destination_city_name": "Seoul",
  "destination_city_code": "SEL",
  "dates": {
    "start": "2025-11-10",
    "end": "2025-11-25"
  },
  "pax": {
    "adults": 2,
    "kids": 2
  },
  "budget": 5000.0,
  "hotel": {
    "request": false,
    "amenities": [
      "breakfast"
    ],
    "rating": 4
  },
  "car": false,
  "attractions": []
}

```

Requested slots sample 2

```

✓ CAN SEARCH FLIGHTS: True ✓
✈ Searching for flights...
✓ Loaded Amadeus Key: Yes
✓ Loaded Amadeus Secret: Yes
{
  "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T18:20:00",
  "link": "https://www.amadeus.com"
},
{
  "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T16:00:00",
  "link": "https://www.amadeus.com"
},
{
  "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T18:20:00",
  "link": "https://www.amadeus.com"
},
{
  "airline": "KE", "price": 5889.12, "currency": "USD", "departure_time": "2025-11-10T16:00:00",
  "link": "https://www.amadeus.com"
},
{
  "airline": "KE", "price": 6141.12, "currency": "USD", "departure_time": "2025-11-10T06:20:00",
  "link": "https://www.amadeus.com"
}
✕ Found cheapest flight: KE for $5889.12

```

Flight API
sample 2

```

✓ HOTEL SEARCH IS SKIPPED ✓
✓ CAN SEARCH ATTRACTIONS: True ✓
✈ Searching for attractions...
✓ Loaded Amadeus Key: Yes
✓ Loaded Amadeus Secret: Yes
✓ Found and processed 3 attractions.
{
  "name": "SEOUL Mobile Public Transport Card for Subway and Bus", "description": "",
  "link": "https://amadeus.booking.holibob.tech/product/9e9df5d9-a462-4db0-b336-73901b"
},
{
  "name": "Blood & Tears: Korea Independence & Resistance Dark History Tour", "description": "USD",
  "link": "https://amadeus.booking.holibob.tech/product/92acc3c1-2717-4d3c-"
},
{
  "name": "Nami Island and Petite France Full-Day Group Tour from Seoul", "description": "USD",
  "link": "https://amadeus.booking.holibob.tech/product/8a4d8d7e-e333-43a1-a2d4"
}
Found 3 attractions.

```

Attractions API
sample 2

API Figure 3. Log to check/debug that APIs are working properly and following the principles 2

4. Database and Backend API Management (Ayo)

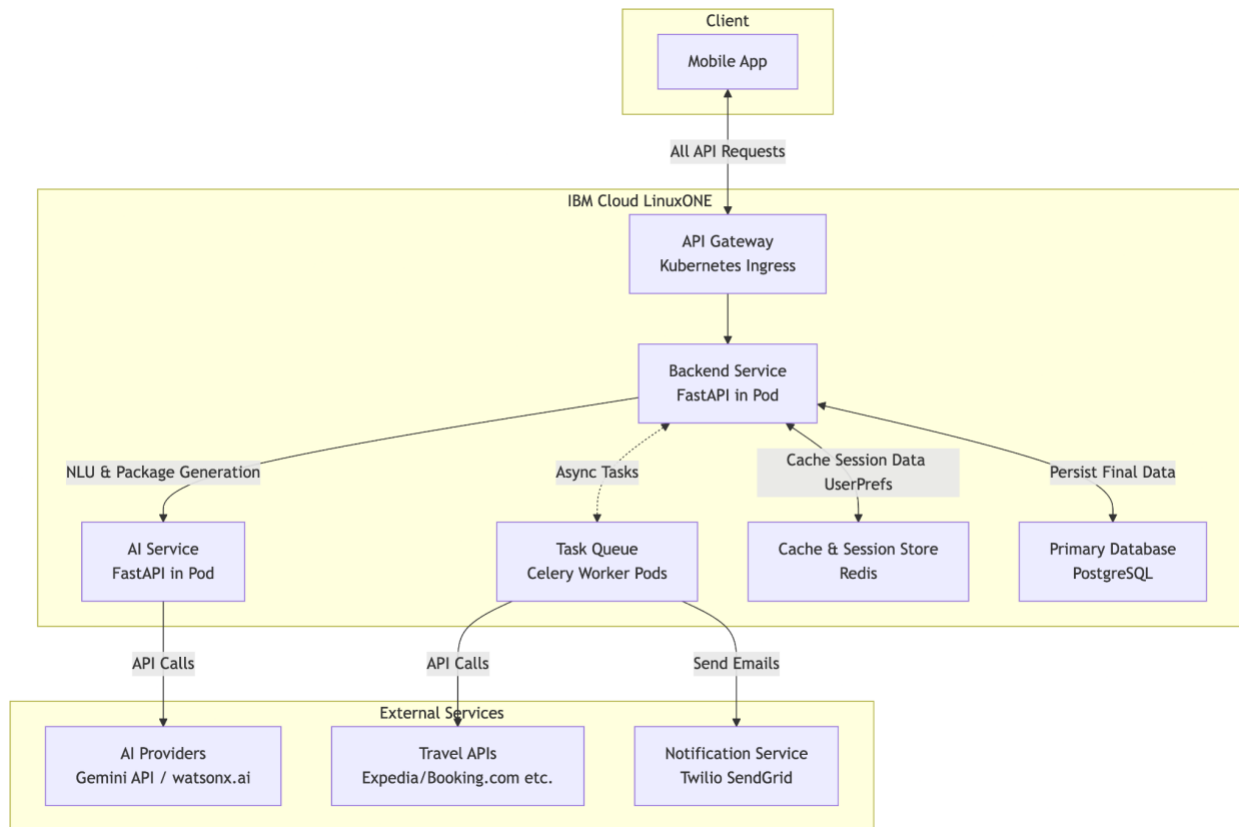


Figure 1 - High-Level System Architecture

Description: Diagram showing the overall system architecture with frontend, backend, database, and external service interactions.

Key components and their interactions

1. **Client & Entry Point**, which is the mobile app that sends HTTPS requests and maintains WebSocket connections for real-time updates.

2. API Gateway & Ingress (IBM Kubernetes Service)

- Tech: NGINX Ingress Controller on IKS.
- Role: Single public entry point handling SSL termination, routing (/api/*, /ws), and load balancing.

- **Data Flow:** Routes external traffic to respective backend or WebSocket pods.

3. Core Backend Service (FastAPI)

- **Role:** Central API handling authentication, trip and chat management, and data persistence.
- **Data Flow:** Receives client requests → validates token via Redis session → interacts with PostgreSQL for persistent storage → returns structured response.
- **Stateless:** All session data is stored in Redis for scalability.

4. Real-Time Communication Service (FastAPI + WebSockets)

- **Role:** Manages persistent WebSocket connections for live updates.
- **Data Flow:** Receives task completion signals (e.g., from Celery) → notifies client instantly through active WebSocket channels.

5. AI Adapter Service

- **Role:** Unified interface for multiple AI providers (Gemini, watsonx.ai).
- **Data Flow:** Backend sends formatted user context → adapter calls AI API → standardizes and returns structured JSON response.
- **Advantage:** Decouples AI logic from core backend.

6. Data Layer & Caching

- **PostgreSQL:** Stores persistent entities (users, trips, packages).
- **Redis:** Manages active sessions, cached states, and Celery message queues.
- **Data Flow:** FastAPI ↔ Redis (session/cache) ↔ PostgreSQL (final persistence).

7. Asynchronous Processing Layer (Celery Workers)

- **Package Builder Worker:** Executes long-running tasks like building trip packages using external travel APIs; saves results to PostgreSQL; triggers WebSocket notifications.

- Notification Worker: Sends transactional emails via SendGrid.
- Data Flow: Backend enqueues task → Celery executes → updates DB → notifies client.

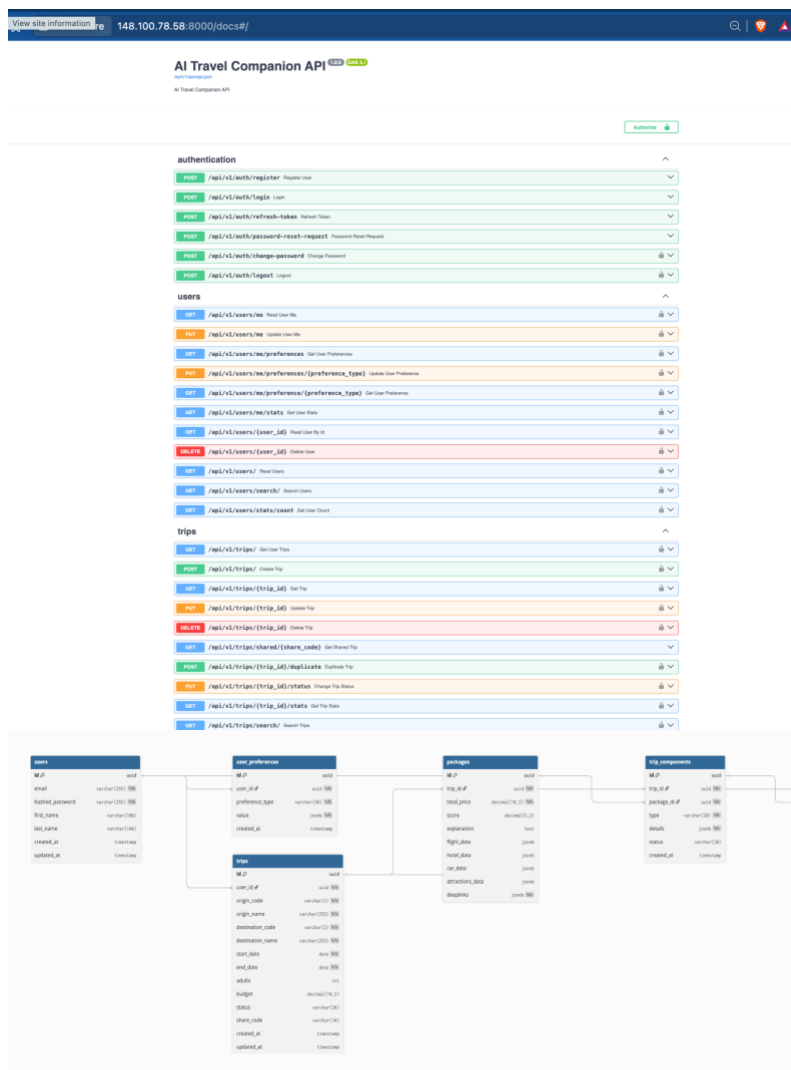
8. External Services

- AI Providers: Handle language understanding and response generation.
- Travel APIs: Supply real-time flight, hotel, and attraction data.
- Notification Service: Sends user emails and alerts.

Key Design Principles Achieved

1. **Scalability:** Every component (FastAPI services, AI Services, Celery workers) is stateless and can be horizontally scaled by adding more Kubernetes Pods. The use of Redis for session storage enables this.
2. **Low Latency:** The user's chat experience is fast because the common path (sending a message and receiving a reply) involves only the Backend Service, Redis, and the AI Adapter. The expensive work is done asynchronously.
3. **Separation of Concerns:** Each service has a single responsibility (e.g., API handling, AI integration, real-time comms, background processing). This makes the system easier to develop, test, and maintain.
4. **Resilience:** If the Package Builder worker fails or is slow, it does not impact the user's ability to continue chatting. The task will remain in the queue and be processed eventually.

5. **IBM Cloud Native:** The architecture leverages managed **IBM** Cloud services (**IKS**, Databases for **PostgreSQL/Redis**), reducing operational overhead and ensuring compatibility with the **LinuxONE** environment.



```

psql (13.22)
Type "help" for help.

twos_db=# \dt
               List of relations
 Schema |          Name          | Type  | Owner
-----|-----|-----|-----
 public | booking_references    | table | postgres
 public | packages              | table | postgres
 public | trip_components       | table | postgres
 public | trips                 | table | postgres
 public | user_preferences      | table | postgres
 public | users                 | table | postgres
(6 rows)

twos_db=#

```

Figure 4 - Sample Database Table

Description: Example of database tables with sample data.

Full User Authentication & Security Flow With JWT, with full security measures implemented

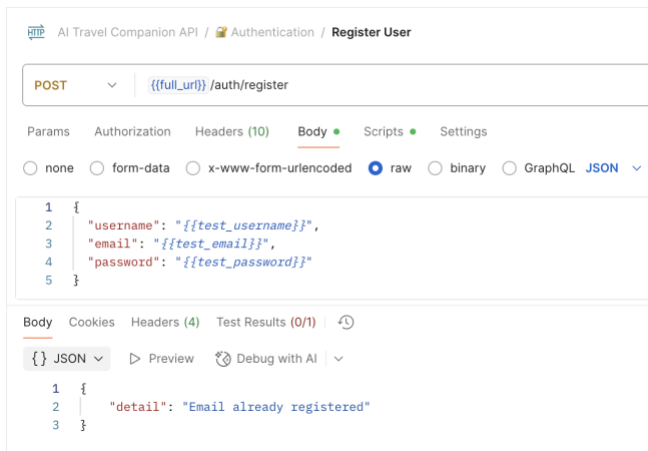


Figure 5 - User Register Flow

POST /api/v1/auth/register - User registration (requires username, email, password only)

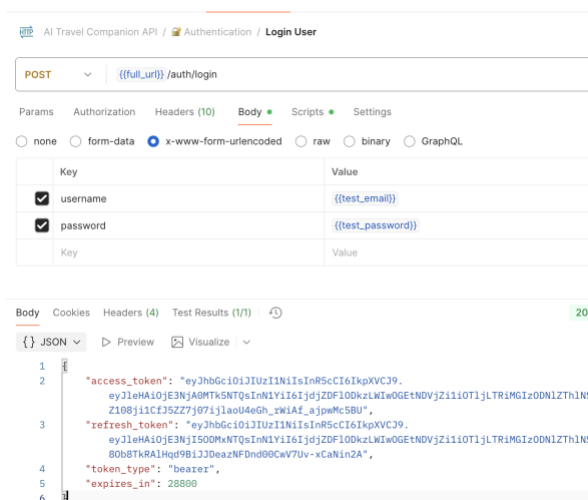


Figure 6 - User Login Flow

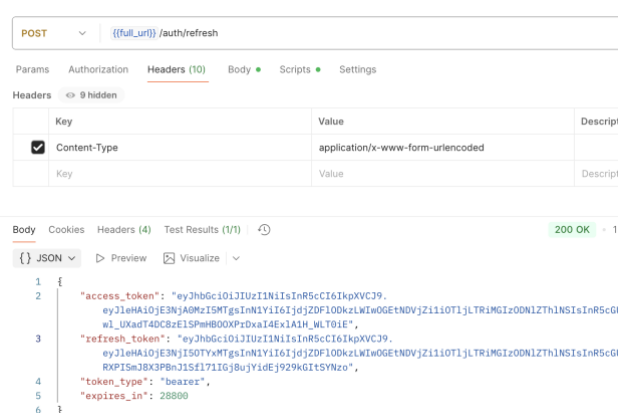


Figure 7 - User Token Management Flow

POST /api/v1/auth/refresh - Access tokens expire in 8 hours, refresh tokens in 30 days

POST /api/v1/auth/login - Uses email/username and password to generate access and refresh tokens

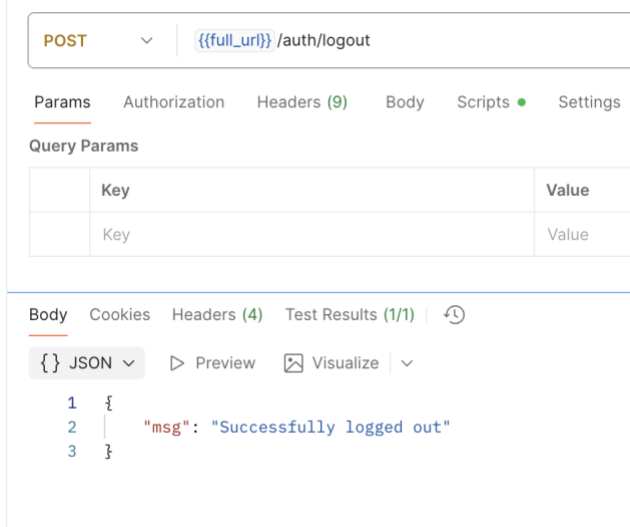


Figure 8 - User Logout Flow

POST /api/v1/auth/logout - Server-side logout tracking for audit purposes(frontend will handle logout by removing the user token from the browser session)

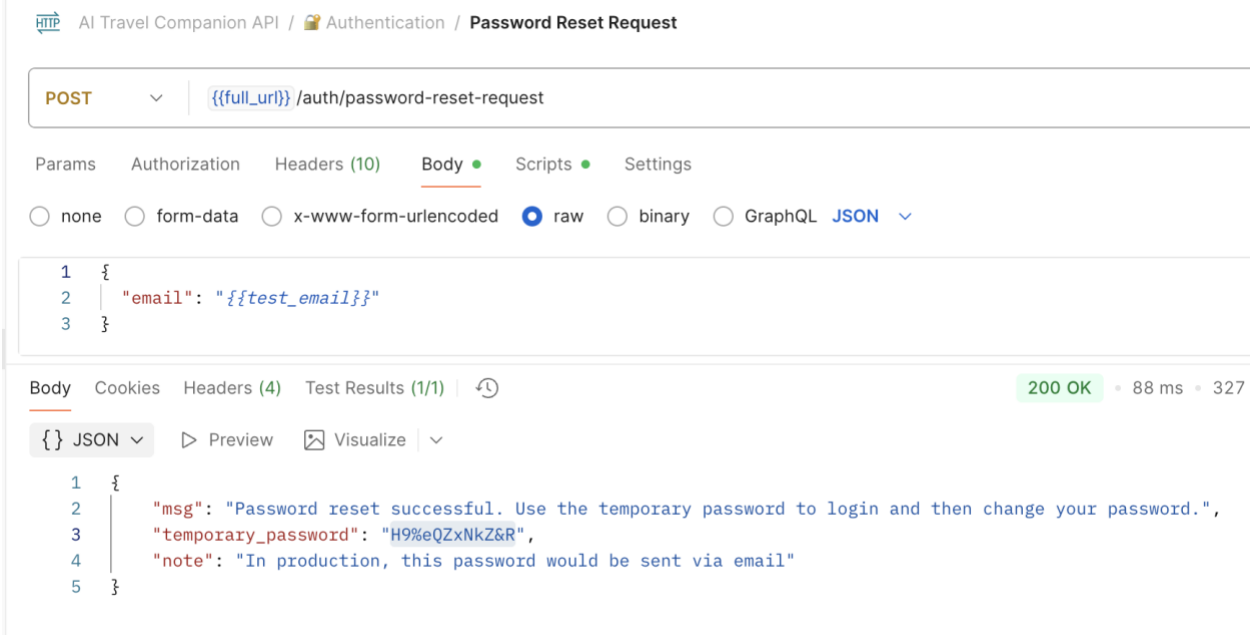


Figure 9 - User Password Reset Flow

POST /api/v1/auth/password-reset-request - Generates temporary passwords for secure password recovery

Password Reset Flow(Production)

1. User requests a password reset via email
2. The system generates a temporary password
3. User logs in with a temporary password to get an access token

4. User changes password using the change-password endpoint with an access token

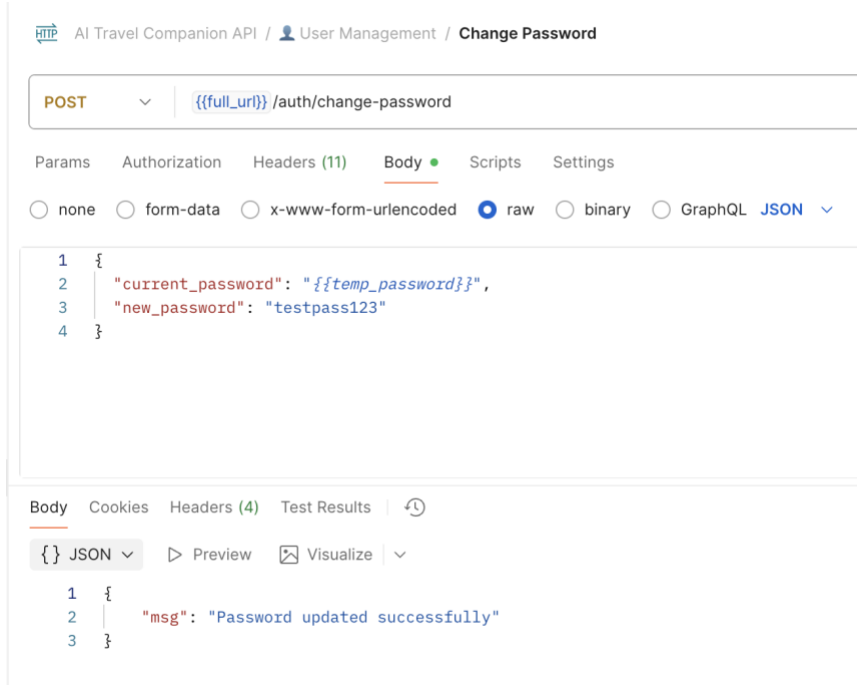


Figure 10 - User Password Change Flow

POST /api/v1/auth/change-password - Allows users to change passwords with current password verification

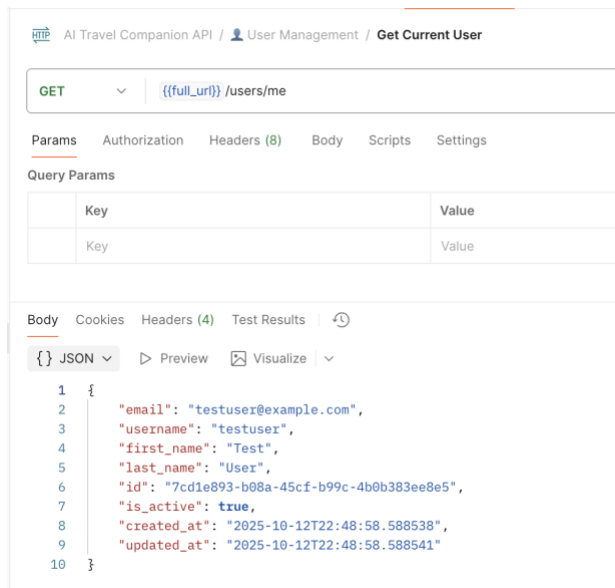


Figure 11 - Verify User is authenticated

GET /api/v1/users/me - Get current user profile

5. Cloud Deployment and Infrastructure (Ayo)



Figure 12 - Docker configuration and services deployed

Containerized Services:

- Backend API: FastAPI-based RESTful service
- Database: PostgreSQL 13 (s390x architecture compatible)
- AI Service: Optional separate container for AI processing
- Redis Service: Optional separate container for AI processing
- Networking: Docker bridge network (ai_travel_network)

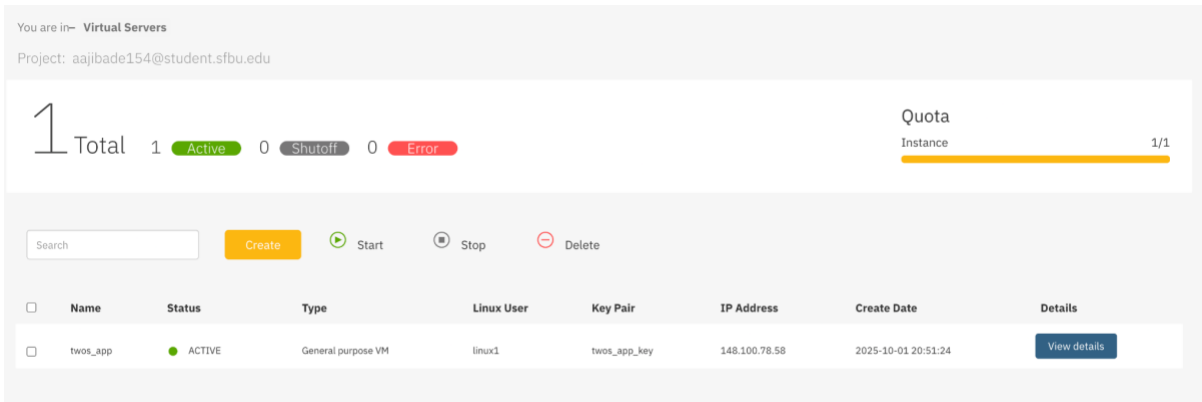


Figure 13 - IBM LinuxOne virtual server

Prerequisites:

- Docker CE for IBM Z/LinuxONE
- Docker Compose
- Sufficient storage (min 20GB recommended)

- Open ports: **8000** (API), **5432** (Database)
- Leverage LinuxONE's vertical scaling capabilities
- Adjust CPU/Memory allocations in the compose file



Figure 14 - TWO App Project structure

Github: https://github.com/AyoyimikaAjibade/ai_travel_companion

Conclusion

We have proposed an AI-powered travel planning application that reduces the effort and time of searching for flights, hotels, rental cars, and attractions manually. The application integrates the natural language processing module and external APIs with backend services to provide optimized results from users' everyday language input. The frontend delivers a clear, structured, and interactive interface, while the backend ensures efficient resource and data handling. Future work will focus on refining the NLP module, expanding API integration, and improving performance to provide travelers with not only a faster and reliable, but also a more personalized planning experience.

References

Booking.com Demand API. (2025). [Booking.com](https://developers.booking.com/demand/docs/open-api/demand-api).

<https://developers.booking.com/demand/docs/open-api/demand-api>

Connect to Amadeus travel APIs | Amadeus for Developers. (n.d.). Amadeus IT Group SA.

<https://developers.amadeus.com/self-service>

Flights - Expedia Group Developer Hub. (2025). [Expediagroup.com](https://developers.expediagroup.com).

<https://developers.expediagroup.com/rapid/flights#step-1---search-available-flight-offers>

<https://dev.to/gbengelebs/netflix-system-design-backend-architecture-10i3>

Swagger-API. (n.d.). GitHub - swagger-api/swagger-ui: Swagger UI is a collection of HTML, JavaScript, and CSS assets that dynamically generate beautiful documentation from a Swagger-compliant API. GitHub. <https://github.com/swagger-api/swagger-ui>

The PostgreSQL Global Development Group. (2025). PostgreSQL: Documentation.

[Www.postgresql.org](https://www.postgresql.org/docs/). <https://www.postgresql.org/docs/>

Welcome to the Tiqets Distributor API | Tiqets for developers. (2025, September 5). [Tiqets.dev](https://developers.tiqets.dev).

<https://developers.tiqets.dev>

https://dbdiagram.io/d/AI_Travel_Companion-68be1d8461a46d388ede1ce4

draw.io

<https://github.com/linuxone-community-cloud/technical-resources/blob/master/faststart/deploy-virtual-server.md>